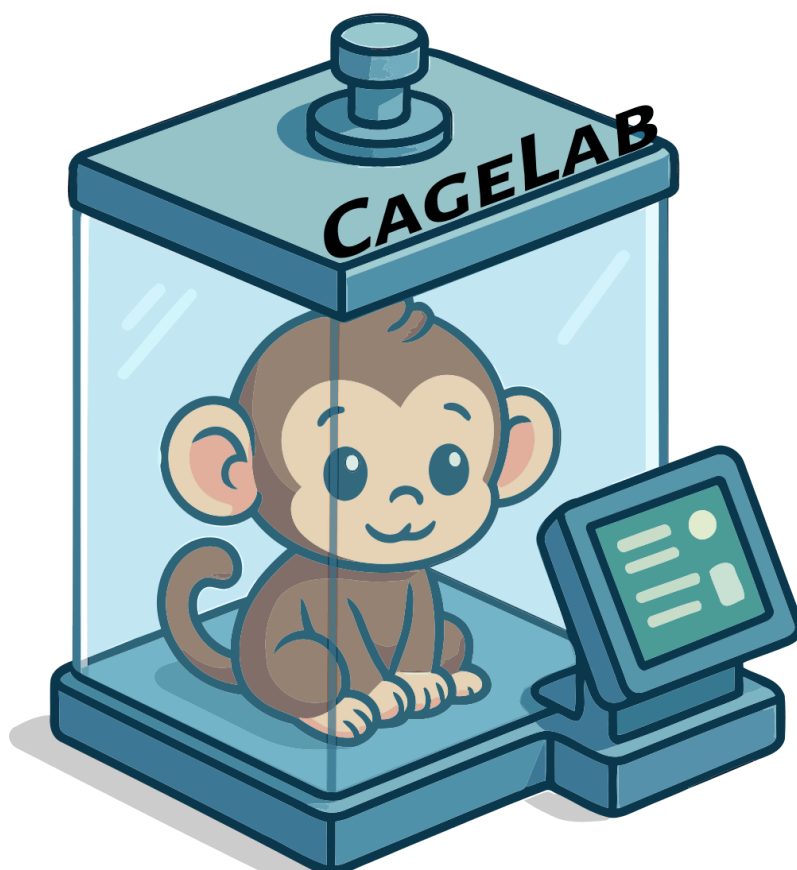

CageLab User Guide

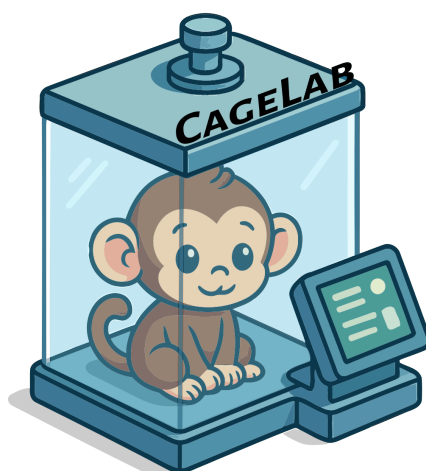
First edition

CageLab Development Team



CageLab User Guide

First edition



This version can be viewed and downloaded free of charge for personal use only. It must not be redistributed, sold, or used in derivative works.

© CageLab Development Team, 2026.

Table of contents

1. Introduction	9
1.1. The Big Picture	10
1.2. Key Components at a Glance	11
1.3. Architecture Deep-Dive: HTTP API + ØMQ Bridge	11
1.4. Supported Hardware	12
2. Dependencies & Installing	13
2.1. Software Stack Overview	13
2.2. CageLab-Box Installation	13
2.2.1. Step 1: Bootstrap	14
2.2.2. Step 2: Run makelinks.sh	14
2.2.3. Step 3: Verify Services on CageLab	14
2.3. Control-PC Installation	15
2.3.1. MATLAB Dependencies	15
2.3.2. Code Repository Structure	15
3. Network Setup	17
3.1. SSH Configuration	17
3.1.1. Creating SSH Keys	18
3.2. NetBird VPN	18
3.2.1. Installation	18
3.2.2. Device Registration	18
3.2.3. Managing the Mesh	18
3.2.4. SSH Config (~/.ssh/config)	19
3.3. NoMachine Remote Desktop	19
3.3.1. Installation	19
3.3.2. Connection	20
4. Using Ansible for Group Maintenance	21
4.1. Setup	21
4.2. The Inventory	21
4.3. Playbook Reference	22
4.3.1. Example Commands	23
4.3.2. The Update Playbook in Detail	23
5. Managing Running Services	25
5.1. Service Architecture	25
5.2. Service List	26

5.3. Convenience Scripts	26
5.3.1. Usage Examples	26
5.4. Manual systemctl Commands	27
5.5. cagelab-monitor with tmuxp	27
5.6. touchscreen Management	28
5.7. OBS Recording Control	28
5.8. Data Management: cagepush and cagepull	29
5.8.1. cagepush — Push Code to Boxes	29
5.8.2. cagepull — Retrieve Data from Boxes	29
6. Using the CageLab GUI	31
6.1. GUI Overview	32
6.1.1. Remote Control	32
6.1.2. Task Selector	32
6.1.3. Session Info	32
6.1.4. Alyx Integration	32
6.2. Running an Experiment	33
6.3. Advanced Features	33
6.3.1. Multiple Box Control	33
6.3.2. Desktop Control Commands	33
6.3.3. Timing Verification	34
7. Using Alyx with CageLab	35
7.1. BEFORE STARTING: Setting up the Database	35
7.2. CageLab: (1) Logging in and running queries	36
7.3. CageLab: (2) Running a Session	38
8. CageLab Tasks	39
8.1. Task Architecture	39
8.2. Task Catalog	40
8.2.1. Touch Training (startTouchTraining)	40
8.2.2. Match-to-Sample (startMatchToSample)	41
8.2.3. Intra/Extra-Dimensional Set Shifting (startIED)	42
8.2.4. Visual Oddball (startVisualOddball)	42
8.2.5. Odd-One-Out / Things Task (startThings)	43
8.2.6. Drag Categorisation (startDragCategorisation)	43
8.2.7. IED Morphobes Variants	43
8.3. The theConductor Command API	43
8.4. cogmoteGO HTTP API	44

9. Troubleshooting	45
9.1. Service Issues	45
9.1.1. cogmoteGO Won't Start	45
9.1.2. theConductor Won't Start	45
9.1.3. Service Dependencies Fail	46
9.2. Network Issues	46
9.2.1. Cannot SSH to Box	46
9.2.2. ØMQ Connection Fails	46
9.3. Task Issues	47
9.3.1. Task Crashes Mid-Session	47
9.3.2. Touchscreen Not Responding	47
9.3.3. OBS Streaming Fails	47
9.4. Display Issues	48
9.4.1. PTB Sync Failures	48
9.5. Log Locations	48
A. Appendix	49
A.1. File and Directory Layout	49
A.2. systemd Service Dependency Graph	50
A.3. Network Port Reference	51
A.4. Key Environment Variables	51
A.5. CageLab Version History	51
A.6. Further Reading	51
B. Index	53

Figure 1.1. Main software components for Control and Communication	10
Figure 1.2. Message Passing	12
Figure 2.1. Installation requirements	13
Figure 3.1. Multi-CageLab communication	17
Figure 5.1. Service relationships	25
Figure 5.2. CageLab Monitor in tmux	27
Figure 6.1. Main CageLab GUI	31
Figure 6.2. GUI Panels	32
Figure 6.3. Alyx Session Setup	33
Figure 7.1. alt text	36
Figure 7.2. Alyx IP address	36
Figure 7.3. Data repo	36
Figure 7.4. Logged In	37
Figure 7.5. Query 1	37
Figure 7.6. Query 2	37
Figure 7.7. Enable Alyx	38
Figure 7.8. Session	38
Figure 7.9. Sessions	38
Figure 8.1. General Trial Loop	40
Figure A.1. systemd boot dependencies	50

Table 1.1. Key components	11
Table 2.1. MATLAB toolboxes	15
Table 2.2. The Git Repositories	16
Table 4.1. Default groups	22
Table 4.2. Ansible playbooks	22
Table 5.1. The main services that run CageLab systems	26
Table 5.2. CageLab scripts	26
Table 5.3. tmux keyboard commands	28
Table 6.1. Remote commands	34
Table 8.1. Touch training	40
Table 8.2. Touch parameters	41
Table 8.3. Match-to-sample variants	41
Table 8.4. MTS parameters	41
Table 8.5. IED Types	42
Table 8.6. IED parameters	42
Table 8.7. Visual Oddball parameters	43
Table 8.8. Things Task parameters	43
Table 8.9. Loop commands	44
Table 8.10. REST API endpoints	44
Table 9.1. Log locations	48
Table A.1. Ports used	51
Table A.2. ENV vars	51
Table A.3. Versions	51

Chapter 1

Introduction

CageLab is a modular, network-distributed system for running remote neuroscience behavioural experiments. It separates the **Control-PC** (the experimenter's workstation, usually in an office) from the **CageLab-Box** (the cage-attached computer driving stimuli, touch, reward, and video recording), allowing a single researcher to manage many subjects across multiple cages from one place.

Content

1.1. The Big Picture	10
1.2. Key Components at a Glance	11
1.3. Architecture Deep-Dive: HTTP API + ØMQ Bridge	11
1.4. Supported Hardware	12

1.1. The Big Picture

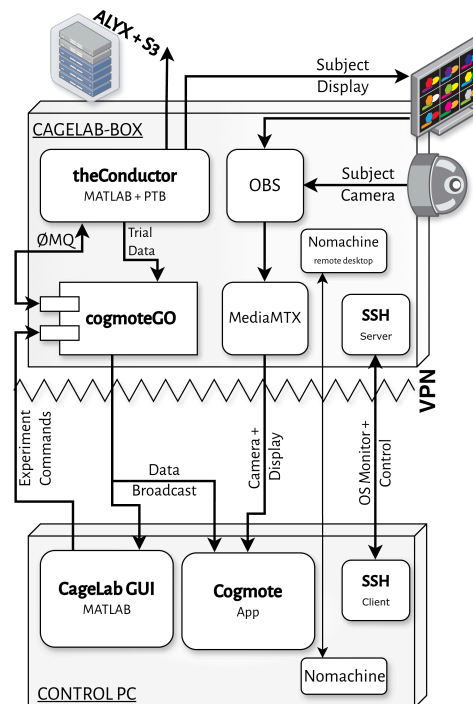


Figure 1.1 – Main software components for Control and Communication

CageLab is a combination of hardware and software. The software uses several core software components, connected over a local network or VPN.

The system is built on three principles:

1. **Distributed architecture** — the experimenter designs tasks and monitors behaviour from their Control-PC; the CageLab-Box runs the experiment locally with deterministic timing via Psychtoolbox (PTB). If necessary a researcher can run the control interface and experiment from the same system (useful for debugging & development for example, or if there is no Control-PC available).
2. **HTTP API + ØMQ bridge** — all external commands from the Control-PC arrive via **cogmoteGO**'s HTTP REST API; internally, **cogmoteGO** bridges these commands to **theConductor** over a local ZeroMQ (ØMQ) connection.
3. **Automation-first** — Ansible playbooks bootstrap, update, and maintain groups of CageLab-Boxes simultaneously; systemd services keep critical processes alive and self-healing.

1.2. Key Components at a Glance

Table 1.1 – Key components

Component	Role	Language	Runs On
CageLab GUI	Experiment design, control, and monitoring	MATLAB	Control-PC
cogmoteGO	HTTP API / ØMQ bridge, WebRTC streaming	Go	CageLab-Box
theConductor	ØMQ REP server, receives commands and launches tasks	MATLAB	CageLab-Box
+cltasks	Behavioural task library (MTS, IED, Odd-ball, etc.)	MATLAB	CageLab-Box
OBS Studio	Video recording of experimental sessions	C++ (Flatpak)	CageLab-Box
MediaMTX	RTSP/WebRTC media server for low-latency streaming	Go	CageLab-Box
SSH	remote access,	System	Control-PC
Ansible	Group automation, updates, and maintenance	Python	Control-PC
NetBird	WireGuard-based VPN mesh for cross-site connectivity	Go	Both

1.3. Architecture Deep-Dive: HTTP API + ØMQ Bridge

`cogmoteGO` exposes two interfaces:

- **External:** an HTTP REST API on port `9012`, used by the Control-PC (CageLab GUI, curl, Ansible). Full API documentation: cogmotego.apifox.cn/.
- **Internal:** a local ØMQ connection to `theConductor`, used only within the CageLab-Box.

The communication stack works as follows:

1. `cogmoteGO` starts first, binding its HTTP API on port `9012` and preparing an internal ØMQ channel for local communication.
2. `theConductor` starts next. It opens a **REP** ØMQ socket on port `6666` and calls `cogmoteGO`'s HTTP API (`POST /api/cmds/proxies`) to register itself as a **command proxy** named `matlab`. This tells `cogmoteGO` where to forward incoming commands.
3. A handshake occurs: `cogmoteGO` sends `{"request": "Hello"}` over the local ØMQ channel; `theConductor` responds `{"response": "World"}`. The proxy is now active.
4. When the Control-PC sends a command (e.g., to run a task), it POSTs JSON to `cogmoteGO`'s HTTP API. `cogmoteGO` bridges the command to `theConductor` via the internal ØMQ proxy, waits for the reply, and returns it as the HTTP response.

5. `theConductor`'s `process()` loop polls the `ØMQ` socket for incoming messages, dispatches commands (`run`, `echo`, `gettime`, `commandlist`, etc.), and sends structured replies back through the proxy.

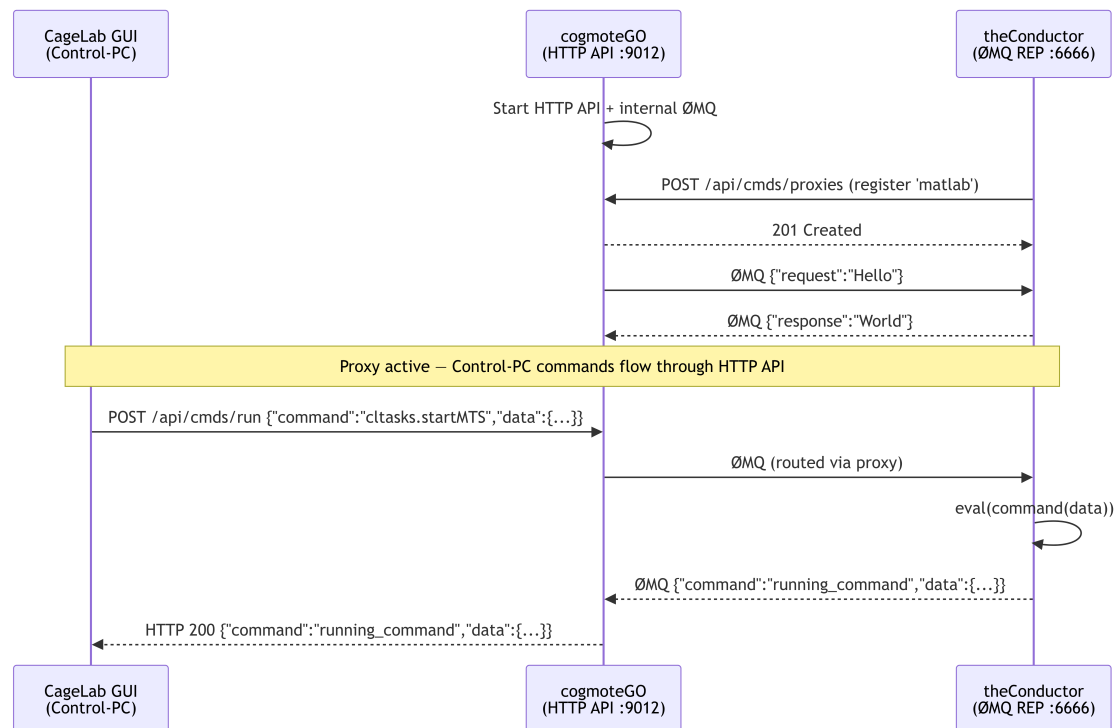


Figure 1.2 – Message Passing

1.4. Supported Hardware

- **Touchscreens:** ILITEK-TP (tested) and other X11-compatible touch panels (controlled via `toggleInput` and `touchManager`)
- **Reward Pumps:** PTBSimia-managed peristaltic/syringe pumps
- **Audio:** Standard sound cards via Opticka's `audioManager`
- **Camera:** Anything compatible with OBS Studio (hardware-accelerated encoding (VA-API/NVENC))
- **Screens:** Standard 60–240 Hz LCD/OLED panels or dedicated research panels via DisplayPort/HDMI (FreeSync/G-Sync compatible) thanks to PTB.

Chapter 2

Dependencies & Installing

This chapter covers software installation for both the **CageLab-Box** (the remote experiment computer/kiosk) and the **Control-PC** (the researcher's workstation).

Content

2.1. Software Stack Overview	13
2.2. CageLab-Box Installation	13
2.3. Control-PC Installation	15

2.1. Software Stack Overview

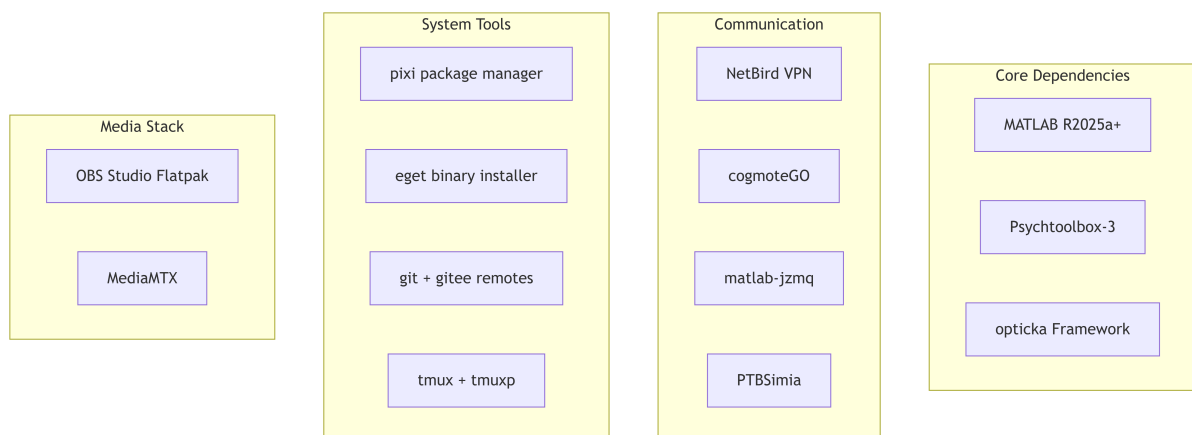


Figure 2.1 – Installation requirements

2.2. CageLab-Box Installation

The CageLab-Box runs **Ubuntu Linux** (24.04 LTS or later) with the **i3 window manager** for a min-

imal, uncomposed X11 environment that has two benefits: (1) PTB timing precision is improved¹; (2) i3 has no touchable GUI, stopping subjects from manipulating any desktop widgets when no task is running.

2.2.1. Step 1: Bootstrap

For a fresh system, we have a bootstrap script that will install dependencies and set up the correct file locations. Clone the CageLab-Code repository and run the bootstrap script:

```
mkdir -p ~/Code
cd ~/Code
git clone --recurse-submodules https://gitee.com/CogPlatform/CageLab-Code.git
cd CageLab-Code/setup
./bootstrap.sh
```

The bootstrap script performs the following: - Installs essential APT packages (git, curl, zsh, i3, tmux, flatpak, etc.) - Installs `pixi` (cross-platform package manager) - Installs NetBird VPN - Optionally: installs MATLAB via MathWorks Package Manager (MPM) - Installs `cogmoteG0` via its install script - Installs NoMachine remote desktop - Installs `eget` and MediaMTX - Clones all core repositories from Gitee - Sets up the PTB/Opticka MATLAB path - Runs `makelinks.sh` to symlink scripts, configs, and systemd services

2.2.2. Step 2: Run makelinks.sh

It is recommended to regularly run `makelinks.sh`, and especially after any code changes:

For CageLab:

```
~/Code/CageLab-Code/setup/makelinks.sh
```

For Control-PC:

```
~/Code/CageLab-Code/setup/makelinks.sh -c
```

This symlinks ZSH configs, script files, systemd services, SSH config, i3 config, tmux config, and the pixi manifest. Use the `-c` flag for Control-PC to ensure Ansible config is correctly linked.

2.2.3. Step 3: Verify Services on CageLab

```
systemctl --user status cogmoteG0.service
systemctl --user status theConductor.service
systemctl --user status mediamtx.service
systemctl --user status obs.service
```

¹Composited desktop environments (GNOME, KDE) introduce unpredictable frame drops and timing jitter. i3 is a tiling window manager with no compositor, ensuring PTB can take full control of the display.

2.3. Control-PC Installation

On the Control-PC, you can install the same services as CageLab-Box if you want to perform any testing, however in general these are not necessary. We use Ubuntu, and our scripts are designed for Posix systems (any Linux, macOS), so some scripts will need to be modified (or run using WSL) on Windows. Running Linux everywhere makes problem solving a bit smoother, and gets stuff used to Linux and thus able to better support the remote system. You typically need:

1. **MATLAB** with the same toolboxes as the CageLab-Box (Curve Fitting, Instrument Control, Optimization, Parallel Computing, Signal Processing, Statistics and Machine Learning).
2. **The CageLab GUI** (part of CageLab-Code, running inside MATLAB).
3. **Ansible** for group management:

```
uv tool install ansible
ansible-galaxy collection install ansible.posix
```

4. **SSH keys** configured for Control-PC and each CageLab-Box (see [Network Setup]).

2.3.1. MATLAB Dependencies

Most of these MATLAB toolboxes are not strictly required but recommended for full CageLab operation:

Table 2.1 – MATLAB toolboxes

Toolbox	Purpose
MATLAB	Core runtime
Curve Fitting Toolbox	Data fitting for analysis
Instrument Control Toolbox	Hardware communication
Optimization Toolbox	Parameter optimization
Parallel Computing Toolbox	Multi-core task execution
Signal Processing Toolbox	Signal analysis
Statistics and Machine Learning Toolbox	Data analysis, classification

All are installable via MPM on Linux:

```
products='MATLAB Curve_Fitting_Toolbox Instrument_Control_Toolbox
Optimization_Toolbox Parallel_Computing_Toolbox Signal_Processing_Toolbox
Statistics_and_Machine_Learning_Toolbox'
mpm install --release=R2025b --products=$products
```

2.3.2. Code Repository Structure

All CageLab code is version-controlled via Git, synced from Github (where we do main development) to Gitee (accessible from behind the Great Firewall). The Gitee repositories:

Table 2.2 – The Git Repositories

Repository	URL	Purpose
CageLab-Code	https://gitee.com/CogPlatform/CageLab-Code.git	Main CageLab codebase
Psychtoolbox	https://gitee.com/CogPlatform/Psychtoolbox.git	PTB-3 fork
opticka	https://gitee.com/CogPlatform/opticka.git	Core stimulus/task framework
matmoteGO	https://gitee.com/CogPlatform/matmoteGO.git	MATLAB ØMQ client
PTBSimia	https://gitee.com/CogPlatform/PTBSimia.git	Reward pump interface
matlab-jzmq	https://gitee.com/CogPlatform/matlab-jzmq.git	Java ØMQ bindings for MATLAB
PacmanTask	https://gitee.com/CogPlatform/PacmanTask.git	Task-specific code

Chapter 3

Network Setup

Content

3.1. SSH Configuration	17
3.2. NetBird VPN	18
3.3. NoMachine Remote Desktop	19

CageLab requires reliable network connectivity between the Control-PC and all CageLab-Boxes. This chapter covers the three networking layers: NetBird VPN, SSH, and NoMachine remote desktop.

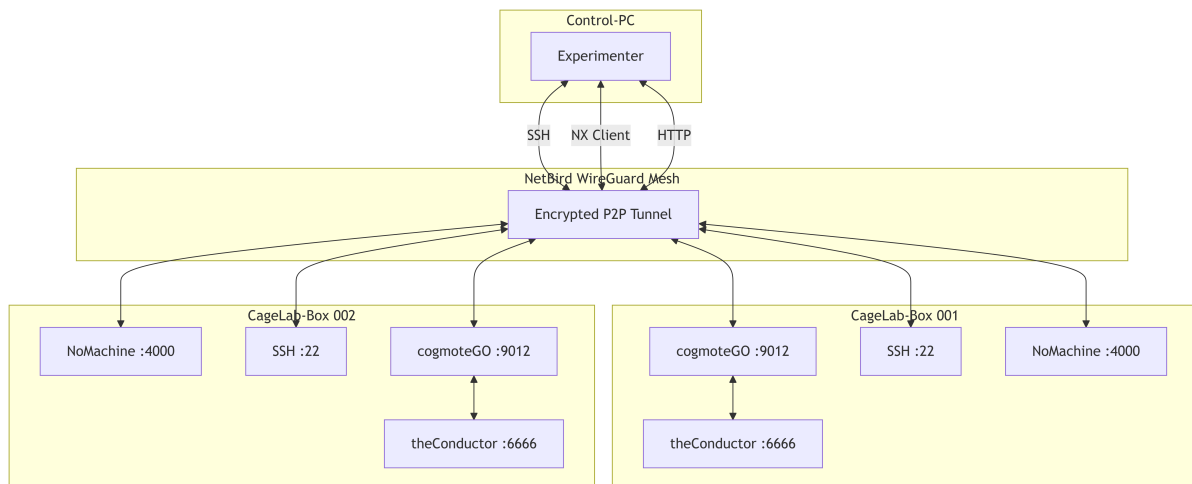


Figure 3.1 – Multi-CageLab communication

3.1. SSH Configuration

SSH is a primary command-line interface to each CageLab-Box. Key-based authentication avoids password prompts, and can be used both for `ssh` from the terminal and the visual remote desktop NoMachine.

3.1.1. Creating SSH Keys

SSH keys are more secure than using passwords, and also more convenient. There are two parts to a SSH key, a private and public key. The private key is kept ONLY on the Control-PC and must be kept private and not shared to unauthorised people. The public key should be installed on all CageLabs. On the Control-PC:

```
ssh-keygen -t ed25519 -f ~/.ssh/cagelab -C "cagelab-control"  
ssh-copy-id -i ~/.ssh/cagelab.pub cagelab@x.x.x.x
```

You must edit `/etc/ssh/sshd_config` to ensure only the Netbird IP is accepted using `ListenAddress`, and disable password access with `PasswordAuthentication no`.

3.2. NetBird VPN

NetBird is a WireGuard-based mesh VPN that provides encrypted peer-to-peer connectivity. It is essential when CageLab-Boxes are on a different physical network from the Control-PC (e.g., separate lab networks or across institutions).

3.2.1. Installation

Install NetBird via the official APT repository (automated via Ansible playbook `install_netbird.yaml` or the bootstrap script):

```
curl -fsSL https://pkgs.netbird.io/install.sh | sh
```

3.2.2. Device Registration

Each device must be registered with your NetBird account using a setup key:

```
netbird up --setup-key YOUR_SETUP_KEY
```

The setup key can be stored as the environment variable `NETBIRD_SETUP_KEY` for automated provisioning via Ansible:

```
NETBIRD_SETUP_KEY=your-key ansible-playbook -l cagelab -K ansible/playbooks/  
install_netbird.yaml
```

3.2.3. Managing the Mesh

- **View connected peers:** `netbird status`
- **Access a box via its NetBird IP:** Once connected, each box gets a stable VPN IP (e.g., `100.112.x.x` or IPv6 `fd00:1:1::x`), and a hostname like `cagelab-001.cloud.lab` reachable from any other peer in the mesh.

3.2.4. SSH Config (~/.ssh/config)

`makelinks.sh` symlinks a pre-configured SSH config from `setup/config/sshconfig`. You MUST edit this to be relevant to your devices and network:

```
Host cagelab-001
  HostName cagelab-001.cloud.lab
  User cagelab
  IdentityFile ~/.ssh/cagelab
  IdentitiesOnly yes

Host cagelab-002
  HostName cagelab-002.cloud.lab
  User cagelab
  IdentityFile ~/.ssh/cagelab
  IdentitiesOnly yes
```

Then you can use just the name to `ssh` into the remote system:

```
ssh cagelab-002                # via SSH config
ssh cagelab@192.168.3.22 -i ~/.ssh/cagelab # manual
```

Naming convention: The `.ssh/config` and the ansible inventory file (`ansible/inventory/hosts`) defines several systems. Boxes accessed via DNS names (`*.cloud.lab`) are in groups like `cagelab`; boxes accessed via raw IPs have `i`-suffixed names (e.g., `cagelab-001i`). This dual-entry approach lets SSH/Ansible target the same box over different network paths if necessary, though you should prefer Netbird routes where possible.

3.3. NoMachine Remote Desktop

SSH is very powerful, however sometimes a desktop is required. NoMachine provides a high-performance remote desktop for GUI access to the CageLab-Box. It's useful for troubleshooting PTB rendering issues, configuring OBS, or inspecting the desktop state.

3.3.1. Installation

Via Ansible:

```
ansible-playbook -l cagelab -K ansible/playbooks/install_nomachine.yaml
```

Or manually:

```
curl -L -o /tmp/nomachine.deb https://web9001.nomachine.com/download/9.6/Linux/nomachine_9.6.3_1_amd64.deb
sudo dpkg -i /tmp/nomachine.deb
```

Next you want to edit the `/usr/NX/etc/server.cfg` file and ensure only SSH key authentication is used, and the listen address is your secure IP address:

```
AcceptedAuthenticationMethods NX-private-key  
NXdListenAddress 100.112.x.x
```

To set up the public SSH key, you can copy the SSH key you have already copied to CageLab to also serve login for Nomachine:

```
cp ~/.ssh/cagelab.pub .nx/config/authorized.crt
```

3.3.2. Connection

Use the NoMachine client (available on macOS, Windows, and Linux) and connect to the CageLab-Box's IP/hostname on port `4000`, using the private part of the SSH key you set up. The LightDM autologin playbook (`install_lightdm_autologin.yaml`) ensures the desktop session starts automatically on boot.

Chapter 4

Using Ansible for Group Maintenance

Content

4.1. Setup	21
4.2. The Inventory	21
4.3. Playbook Reference	22

Ansible is an Infrastructure-as-Code tool that lets you run commands, deploy software, and enforce configuration across all your CageLab-Boxes simultaneously. This is essential when managing 5–50+ experimental rigs.

4.1. Setup

On the Control-PC:

```
uv tool install ansible
ansible-galaxy collection install ansible.posix
```

Then link the CageLab Ansible configuration:

```
~/Code/CageLab-Code/setup/makeLinks.sh -c
```

This symlinks `ansible.cfg` to `/etc/ansible/ansible.cfg` and the inventory to `/etc/ansible/hosts`.

4.2. The Inventory

The inventory (`ansible/inventory/hosts`) defines groups of CageLab-Boxes with their network addresses and SSH credentials, for our use for example:

Table 4.1 – Default groups

Group	Typical Use	Access Method
cagelab	Dev + production boxes	DNS *.cloud.lab)
cagelab_ip	Same boxes, raw IPs	IP address
cagelab_songjiang	Songjiang lab boxes	DNS
cagelab_pacman	Pacman task boxes	DNS
cagelab_prisys	Primate Systems boxes	DNS
cagelab_test	Single test/dev box	DNS
admin	Control-PCs, admin hosts	DNS

Edit the inventory to match your own hosts and IPs. You can also add `mac_address` variables for Wake-on-LAN support.

4.3. Playbook Reference

All playbooks live in `ansible/playbooks/`. Run them with:

```
ansible-playbook -l <group> [-K] [-e "variable=value"] playbooks/<name>.yaml
```

Where: - `-l <group>` limits execution to a specific inventory group - `-K / --ask-become-pass` prompts for the sudo password - `-e` passes extra variables

Table 4.2 – Ansible playbooks

Playbook	Purpose	Requires sudo
update.yaml	Full system update: repos, cogmoteGO, pixi, medi-amtx, OBS. Skips if task running.	Yes
reset_code.yaml	Force-reset all code repos to upstream HEAD (discards local changes).	No
check_api_status.yaml	Check if cogmoteGO reports a running task and show latest broadcast data.	No
services-start.yaml	Start all user systemd services.	Yes
services-stop.yaml	Stop all user systemd services.	Yes
chrony_slave.yaml	Configure NTP slave to a master clock. Set <code>chrony_master_ip</code> env var.	Yes
install_i3.yaml	Install i3 window manager.	Yes
install_lightdm_autologin.yaml	Install LightDM with autologin to i3.	Yes
install_netbird.yaml	Install NetBird VPN and optionally provision with setup key.	Yes
install_nomachine.yaml	Install latest NoMachine.	Yes
enable_wol.yaml	Enable Wake-on-LAN on all ethernet interfaces.	Yes
wol.yaml	Send WoL magic packet to wake specified group.	No
setup_matlab_startup.yaml	Configure MATLAB startup script.	No
upgrade-apt.yaml	APT package upgrade only.	Yes
apt_mirror.yaml	Switch APT to a specific mirror.	Yes

4.3.1. Example Commands

Full update on all CageLab boxes (asks for sudo password once):

```
ansible-playbook --limit cagelab --ask-become-pass playbooks/update.yaml
```

Check if any CageLab box is running a task:

```
ansible-playbook -l cagelab playbooks/check_api_status.yaml
```

Stop all services across all boxes:

```
ansible-playbook -l cagelab -K playbooks/services-stop.yaml
```

Wake all pacman boxes:

```
ansible-playbook -e "wol_group=cagelab_pacman" playbooks/wol.yaml
```

4.3.2. The Update Playbook in Detail

The `update.yaml` playbook is the most comprehensive. It:

1. Queries `cogmoteG0`'s status API — if a task is running, it skips the box.
2. Ensures `/usr/local/bin` is writable and home directory ownership is correct.
3. Updates APT cache if stale.
4. Force-pulls all 7 code repositories to latest HEAD (parallel async).
5. Runs `makelinks.sh` to refresh symlinks.
6. Stops CageLab services.
7. Downloads and runs the `cogmoteG0` installer.
8. Updates `pixi` and its global packages.
9. Updates `mediamtx` via `eget`.
10. Updates OBS Studio via Flatpak.
11. Starts CageLab services.

The entire process is idempotent and safe to run regularly (e.g., weekly).

Chapter 5

Managing Running Services

Content

5.1. Service Architecture	25
5.2. Service List	26
5.3. Convenience Scripts	26
5.4. Manual systemctl Commands	27
5.5. cagelab-monitor with tmuxp	27
5.6. touchscreen Management	28
5.7. OBS Recording Control	28
5.8. Data Management: cagepush and cagepull	29

Each CageLab-Box runs several `systemd` user services that must be kept alive for the system to function. This chapter covers the service lifecycle: startup, monitoring, restarting, and stopping.

5.1. Service Architecture

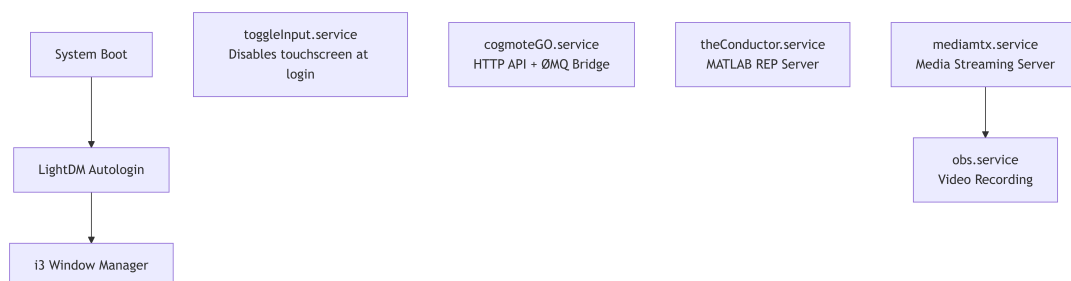


Figure 5.1 – Service relationships

5.2. Service List

Table 5.1 – The main services that run CageLab systems

Service	Type	Purpose	Startup Order
<code>toggleInput.service</code>	Oneshot	Disables touchscreen during boot to prevent subject interaction with OS	1st
<code>cogmoteGO.service</code>	Simple	HTTP API and ØMQ bridge on <code>:9012</code>	2nd
<code>theConductor.service</code>	Simple	MATLAB-based REP server that runs behavioural tasks	3rd
<code>mediamtx.service</code>	Simple	RTSP/WebRTC media server for video streaming	4th
<code>obs.service</code>	Simple	OBS Studio for session video recording	5th

5.3. Convenience Scripts

CageLab provides ZSH convenience scripts in `~/bin/` (symlinked from `scripts/`):

Table 5.2 – CageLab scripts

Script	Action
<code>cagelab-start</code>	[Re]start all services (<code>toggleInput</code> → <code>cogmoteGO</code> → <code>theConductor</code> → <code>mediamtx</code> → <code>obs</code>)
<code>cagelab-stop</code>	Stop all services in reverse order
<code>cagelab-reset</code>	Stop + daemon-reload + restart <code>cogmoteGO</code> and <code>theConductor</code> ; disable touchscreen
<code>cagelab-reset-services</code>	Full service reset: disable old, re-link from repo, re-enable
<code>cagelab-reset-code</code>	Force-reset all code repositories to latest upstream
<code>cagelab-reset-OBS</code>	Stop + restart OBS and <code>mediamtx</code> (for streaming issues)
<code>cagelab-status</code>	Check if <code>cogmoteGO</code> is running and show latest broadcast data
<code>cagelab-update</code>	Full update: stop services → reset code → update <code>cogmoteGO/pixi/mediamtx/flatpak</code> → restart
<code>cagelab-monitor</code>	Launch tmuxp session with CPU monitor + <code>journalctl</code> for <code>cogmoteGO</code> and <code>theConductor</code>

5.3.1. Usage Examples

```
# Quick restart of core services
cagelab-reset

# Full update after code changes
cagelab-update

# Check if everything is running
cagelab-status

# Monitor logs and CPU in real time
cagelab-monitor
```

5.4. Manual systemctl Commands

For fine-grained control, use `systemctl --user`:

```
# Check status
systemctl --user status cogmoteG0.service
systemctl --user status theConductor.service

# View recent logs
journalctl --user -u cogmoteG0.service -n 50 --no-pager

# Follow logs in real time
journalctl --user -f -u theConductor.service

# Restart a single service
systemctl --user restart cogmoteG0.service

# Enable/disable at boot
systemctl --user enable cogmoteG0.service
systemctl --user disable obs.service

# Reload systemd after editing unit files
systemctl --user daemon-reload
```

5.5. cagelab-monitor with tmuxp

The `cagelab-monitor` command launches a tmuxp session with three panes:

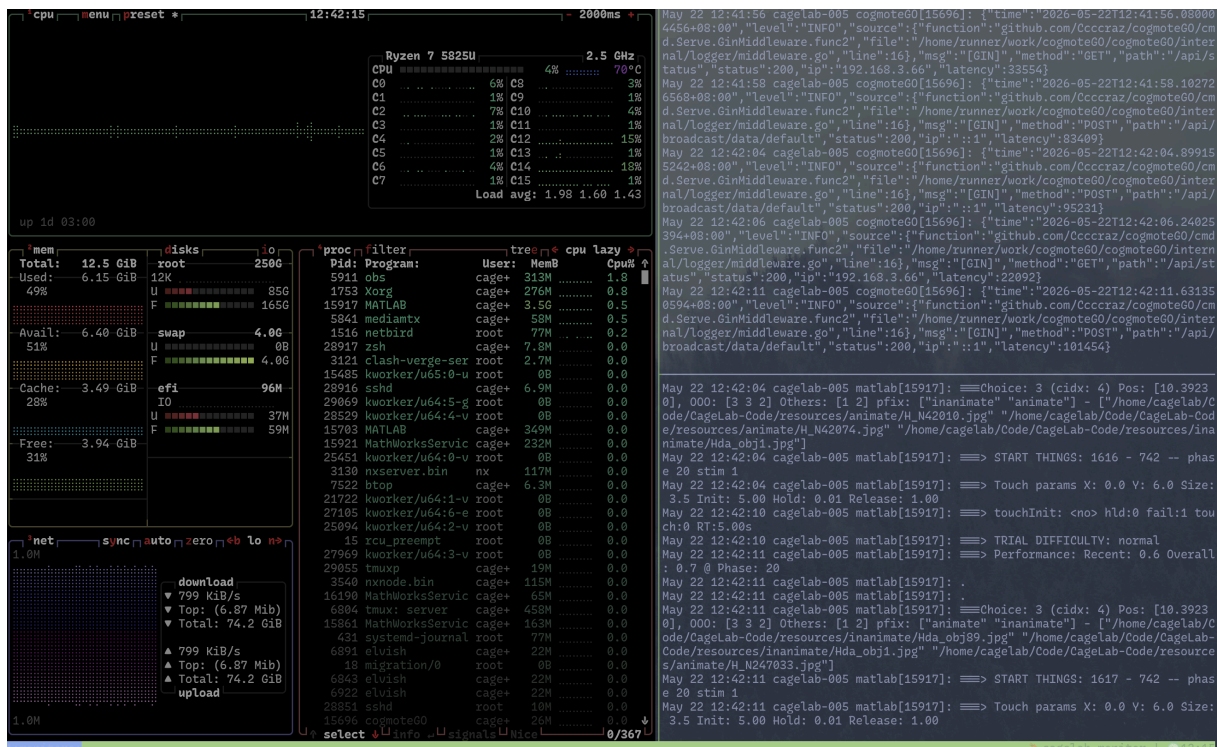


Figure 5.2 – CageLab Monitor in tmux

- **Top-left:** `htop` (CPU/memory monitor)

- **Bottom-left:** `journalctl` following `cogmoteG0` logs
- **Right:** `journalctl` following `theConductor` logs

tmux quick reference (prefix is `Ctrl-a`):

Table 5.3 – tmux keyboard commands

Keystroke	Action
<code>Ctrl-a s</code>	Show session list
<code>Ctrl-a -</code>	Split vertically
<code>Ctrl-a \ </code>	Split horizontally
<code>Ctrl-a x</code>	Kill pane (confirm with <code>y</code>)
<code>Ctrl-a d</code>	Detach from session
<code>Ctrl-a</code> arrow	Navigate panes

The `tmuxp` config is at `~/config/tmuxp/cagelab-monitor.yaml` and is symlinked by `makelinks.sh`.

5.6. touchscreen Management

The `toggleInput` script enables or disables touchscreen devices via `xinput`. This is critical: subjects must not interact with the OS desktop.

```
toggleInput disable          # disable default device (ILITEK-TP)
toggleInput enable           # enable default device
toggleInput disable ILITEK-TP # disable specific device
toggleInput enable "Touchpad" # enable specific device
```

`toggleInput.service` runs at boot to disable the touchscreen. Tasks re-enable it when needed, and `cagelab-reset` disables it afterwards.

5.7. OBS Recording Control

OBS Studio recording can be controlled via its WebSocket API using the `control-OBS.py` script:

```
# Start recording
python3 ~/Code/CageLab-Code/scripts/control-OBS.py start --password YOUR_WS_PASS

# Stop recording
python3 ~/Code/CageLab-Code/scripts/control-OBS.py stop

# Toggle recording
python3 ~/Code/CageLab-Code/scripts/control-OBS.py toggle

# Check status
python3 ~/Code/CageLab-Code/scripts/control-OBS.py status

# Control remote OBS via SSH tunnel
python3 ~/Code/CageLab-Code/scripts/control-OBS.py status \
    --ssh-host cagelab-001.cloud.lab --ssh-user cagelab
```

The password can be set via the `OBS_WS_PASSWORD` environment variable.

5.8. Data Management: `cagepush` and `cagepull`

Two powerful ZSH scripts handle data and file transfer between the Control-PC and CageLab-Boxes:

5.8.1. `cagepush` — Push Code to Boxes

```
# Force push CageLab code from Control-PC to cagelab-001 box (uses rsync with
delete)
cagepush -s cagelab-001.cloud.lab ~/Code/CageLab-Code

# Push multiple projects to multiple hosts in parallel
cagepush -j 4 -s cagelab-001.cloud.lab -s cagelab-002.cloud.lab \
    ~/Code/CageLab-Code ~/Code/opticka

# Dry run (no actual transfer)
cagepush -n -s cagelab-001.cloud.lab ~/Code/CageLab-Code
```

5.8.2. `cagepull` — Retrieve Data from Boxes

```
# Pull saved data from all boxes to local directory
cagepull -l ~/data -s cagelab-001.cloud.lab -s cagelab-002.cloud.lab \
    ~/OptickaFiles/SavedData

# Parallel pull with 4 concurrent transfers
cagepull -j 4 -l ~/experiment-data -s cagelab@cagelab-001 OptickaData
```

Both scripts use `rsync` over SSH with robust retry, progress reporting, `--mkpath` auto-detection, and optional dry-run mode.

Chapter 6

Using the CageLab GUI

Content

6.1. GUI Overview	32
6.2. Running an Experiment	33
6.3. Advanced Features	33

The CageLab GUI is the experimenter's primary interface for designing experiments, connecting to boxes, and monitoring sessions. It runs inside MATLAB on the Control-PC.

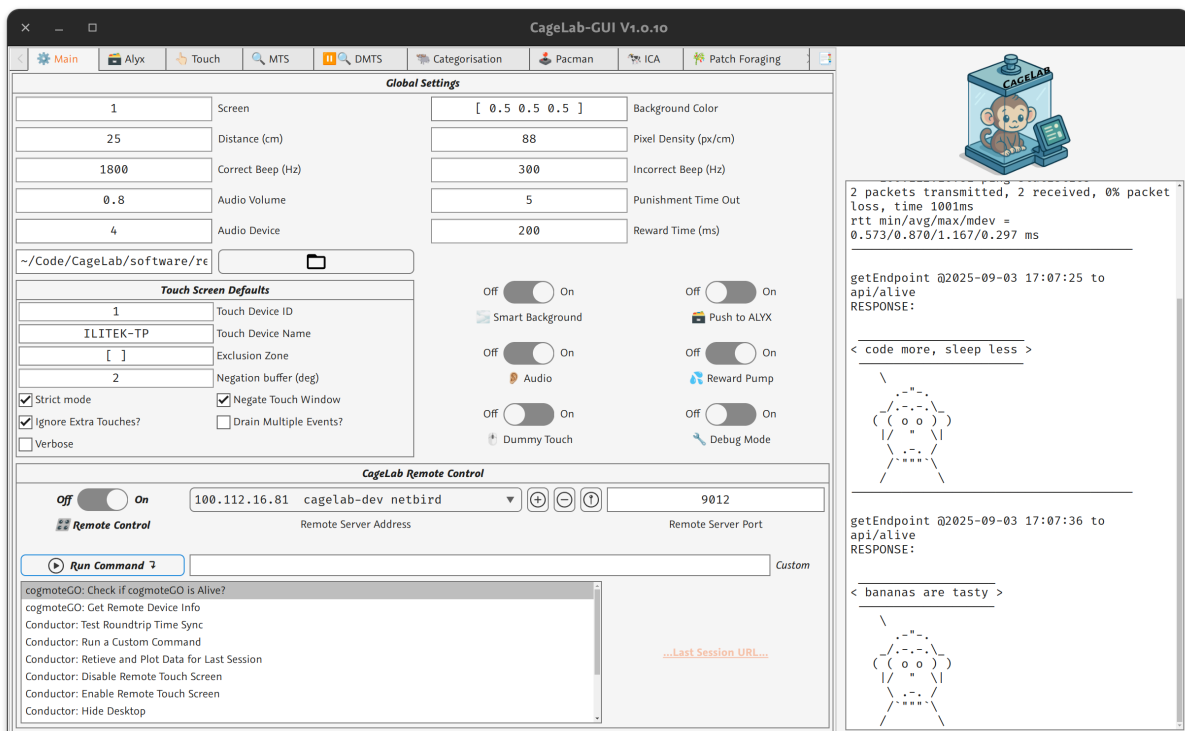


Figure 6.1 – Main CageLab GUI

6.1. GUI Overview

The GUI has several functional areas:

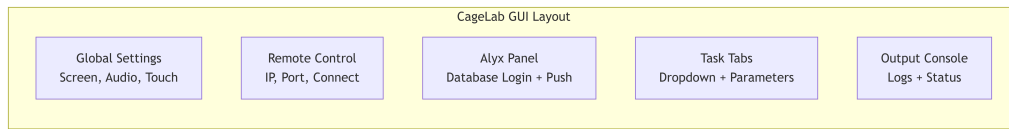


Figure 6.2 – GUI Panels

6.1.1. Remote Control

- **IP Address:** IP or hostname of the CageLab-Box
- **Port:** ØMQ port (default: the port `cogmoteG0` is bound to)
- **Connect / Disconnect:** Establish or tear down the ØMQ connection
- **Test Timing:** Sends a `gettime` command to measure round-trip latency

6.1.2. Task Selector

Selects which behavioural task to run. See [CageLab Tasks] for details on each task. Parameters specific to the selected task appear dynamically, allowing you to configure:

- Stimulus type (disc, picture, shape)
- Sizes, positions, timing
- Distractor count
- Trial parameters

6.1.3. Session Info

Metadata about the experimental session:

- **Subject Name:** Animal/subject identifier
- **Researcher:** Name of the experimenter
- **Lab:** Lab name/location
- **Procedure:** Standard operating procedure
- **Project:** Research project identifier
- **Task Protocol:** Experiment-specific protocol

This metadata is embedded in saved data files and optionally pushed to the Alyx database.

6.1.4. Alyx Integration

See the separate Alyx Guide (`docs/Alyx-Guide.md`) for full details. The GUI panel enables:

1. Login to the Alyx database
2. Query subjects, sessions, and projects
3. Push session metadata and data files via Minio S3

Session Information:	
guiyi	CognitionPlatform
<i>Subject ID</i> 	<i>Lab Name</i> 
Ian	cagelab-002
<i>Researcher</i> 	<i>Experiment Location</i> 
Behavior training/... ▼	Test-Training ▼
<i>Procedure</i>	<i>Project</i>
Cerebral cortex ▼	
<i>Brain Region</i>	<i>Task Protocol</i>

Figure 6.3 – Alyx Session Setup

6.2. Running an Experiment

1. **Connect:** Enter the CageLab-Box IP/port and click Connect.
2. **Configure:** Select a task, adjust parameters, fill in session info.
3. **Start:** Click “Run Task”. The GUI sends a JSON `{"command":"cltasks.startXxx","data":{"...}}` message via ØMQ.
4. **Monitor:** The output console shows log messages. Status updates appear in the status bar.
5. **Stop:** Click “Stop Task” or wait for the task to complete. Data is saved to `~/optickafiles/savedata/` on the CageLab-Box.

6.3. Advanced Features

6.3.1. Multiple Box Control

A single CageLab GUI can connect to many CageLab-Boxes simultaneously. Each connection is independent — you can start a task on Box 1, configure a task on Box 2, and monitor both.

6.3.2. Desktop Control Commands

Via `theConductor`’s command interface, the GUI can:

Table 6.1 – Remote commands

Command	Effect
<code>hidedesktop</code>	Hide the OS desktop (shows solid colour) to prevent subject distraction
<code>showdesktop</code>	Restore the OS desktop
<code>disablesleep</code>	Prevent display sleep/power saving
<code>enablesleep</code>	Re-enable normal display power management
<code>rundemo</code>	Run PTB VBLSyncTest for timing verification

6.3.3. Timing Verification

The `gettime` command performs a round-trip timing test:

1. Sends `GetSecs` value from Control-PC
2. `theConductor` returns its own `GetSecs` and system time
3. Differences reveal network latency and clock drift

Chapter 7

Using Alyx with CageLab

#minitoc

Alyx is an experiment database made by International Brain Lab. While Alyx is mostly focused on neuroscience tasks in mice, it can be generalised to other species as it supports generic JSON fields, and the core structure across species is more similar than the differences anyway. We use TWO services:

- Alyx database — register subjects, labs, experimenters, projects and tasks; each time a session is run record the experiment metadata and link to the binary experiment files.
- S3 data store (**Minio** server) — copy raw binary and text experiment files saved by each session linked to the Alyx session metadata. Searching in Alyx can recover the files saved in the S3 data store.

In the CageLab GUI there are two main interactions with Alyx:

- 1) Login / query the database. Not linked to an experiment session.
- 2) Setup session data and enable CageLab to add the session to the database. The session is added by the remote system, not the control computer.

7.1. BEFORE STARTING: Setting up the Database

!

You **MUST** ensure the Alyx database is setup with the core items you will use to run the session:
Subjects, Users, Labs, Locations, Projects

WELCOME, ADMIN. GO TO SUBJECTS / ACTIONS / DATA / MISC / EXPERIMENT

Home > Subject admin > Subjects

Select subject to change

Search 6 results (Clear search)

Action: ----- Go 0 of 6 selected

	NICKNAME	2	WEIGHT %	BIRTH DATE	1	SEX	ALIVE	# SESS	RESPONSIBLE USER	LAB
☐	guiyi	-	-			M	✓	0	Jiahao	CognitionPlatform
☐	guanguan	-	-			M	✓	0	Jiahao	CognitionPlatform
☐	Cosmos	-	-			U	✓	1	Jiahao	CognitionPlatform
☐	TestSubject	-		March 1, 2023		F	✓	100	Ian	CognitionPlatform

Figure 7.1 – alt text

Before running any sessions, ensure that the items you will use exist in the Alyx database, login to your Alyx web interface (e.g. <http://172.16.102.30:8000>) and edit the database to add the **Subjects**, the **Users** (Researcher), the **Lab**, and the **Projects** the sessions will be linked to.

7.2. CageLab: (1) Logging in and running queries

CageLab allows you to enter your Alyx IP/port and user name and then login:

http://172.16.102.30:8000 ALYX IP & Port

Ian ALYX User

Login Log out

Figure 7.2 – Alyx IP address

There is also a place to enter the Data repo IP and the data repo (bucket) name, and also to be able to edit the Alyx login password and data repo keys:

http://172.16.102.77:90 Alyx Data Repository URL

cognitionplatform Alyx Data-Repository Name

New Alyx Password Change PW

Figure 7.3 – Data repo



These details are stored using MATLAB's `getSecret` and `setSecret` functions and are stored on the local, not remote machine. The remote machine can be set up with its own secrets or when a session runs the local secrets are sent to the remote system at runtime.

Once logged in you will see the query panel become active:

The screenshot shows the CageLab interface after a successful login. At the top, the ALYX IP & Port is `http://172.16.102.30:8000` and the ALYX User is `Ian`. Below this, a green button indicates **LOGGED IN ✓**, and a **Log out** button is available. The ALYX Data Repository URL is `http://172.16.102.77:90` and the name is `cognitionplatform`. A section for changing the password is also present. The **Query Alyx Database:** section is active, showing a query `sessions?subject=test02` and a **Request** button. Below the query, there are input fields for `... weight in kg` and `... water given`, each with a **Weight** or **Water** button.

Figure 7.4 – Logged In

Note that you are logged in from your computer. This is nothing to do with a remote cageLab. By logging in it allows you to run Alyx database queries, for example `sessions?subject=TestSubject` will return all sessions recorded by Alyx for subject `TestSubject` :

The screenshot shows the same interface as Figure 7.4, but with a table of query results displayed. The table has columns for `id`, `subject`, and `start_time`. The results show multiple sessions for the subject `TestSubject`.

id	subject	start_time
b659b25-b1ab-4a11-b923-59b830a21806	TestSubject	2025-12-19T03:15:01
689b886a-587b-4f56-8a06-a72f4a2b6b7b	TestSubject	2025-12-19T10:28:19
8bb8886a-8c91-4c03-bada-76667e9f9aaf	TestSubject	2025-12-19T12:14:30
7ed7d6dd-f2a4-4b6c-8a2a-3980ebaa879b	TestSubject	2025-12-19T14:32:54
4b034d7f-3980-4072-4a94-764c06f0a001	TestSubject	2025-12-19T18:30:23
cbab6d8b-4b1b-4b3f-a38f-ed0e9a27352a	TestSubject	2025-09-09T18:25:3
144f61b3-6665-4943-844e-934f5e9e99f	TestSubject	2025-09-09T17:35:1
841d5b79-5444-44ec-8a7e-14a6dcdb2a75	TestSubject	2025-09-09T17:32:5
7a739df9-fd5d-4451-91cc-39f8db6c45d4	TestSubject	2025-09-09T16:30:0

Figure 7.5 – Query 1

... or for example the lab locations where experiments are run:

The screenshot shows the same interface as Figure 7.4, but with a table of query results displayed. The table has columns for `name`, `lab`, and `json`. The results show various lab locations and their associated data.

name	lab	json
Room-420	CognitionPlatform	
Songjiang-01	CognitionPlatform	
Room-412	CognitionPlatform	
pacman-001	CognitionPlatform	1x1 struct
cagelab-002	CognitionPlatform	1x1 struct
cagelab-001	CognitionPlatform	1x1 struct
cagelab-003	CognitionPlatform	1x1 struct
cagelab-004	CognitionPlatform	1x1 struct
cagelab-005	CognitionPlatform	1x1 struct
cagelab-dev	CognitionPlatform	1x1 struct
pacman-002	CognitionPlatform	1x1 struct
pacman-003	CognitionPlatform	1x1 struct

Figure 7.6 – Query 2

A full list of all the queries you can perform is available from the Alyx database itself: <http://172.16.102.30:8000/docs/>

7.3. CageLab: (2) Running a Session

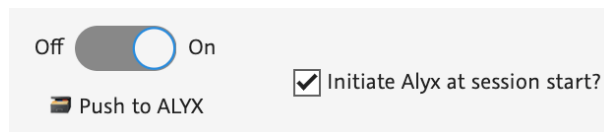


Figure 7.7 – Enable Alyx

When you use CageLab GUI, turn ON push to Alyx. You can also “initiate” Alyx at session start so that Alyx logs a session once it initiates (otherwise it will only register after it is manually ended, thus a crash would not show in Alyx and data be lost).

Then ensure your Session data is accurate and consistent with what is in the database:

Figure 7.8 – Session

Subject, Lab, User and Location MUST already exist in Alyx or else it will crash. Procedure, Project, Brain Region should also be valid (they are populated after logging in to ensure they are valid). Task protocol is optional.

When you run the task, Alyx will be initiated with a new Session, and when the session is stopped / ends, the MAT and LOG files will be uploaded to the S3 data server. You can check in the Sessions view:

Home > Action admin > Sessions

Select session to change

Search 105 results (Clear search)

Action: ----- Go 0 of 50 selected

	SUBJECT	START TIME	1	NUMBER	LAB	3	# DATASETS	TASK PROTOCOL	2	QC	USERS	PROJECT
☐	TestSubject	Dec. 19, 2025, 3:15 a.m.	1		CognitionPlatform		2			PASS	Ian	Things-Training
☐	TestSubject	Dec. 15, 2025, 10:28 a.m.	1		CognitionPlatform		2			PASS	Ian	Things-Training
☐	TestSubject	Dec. 12, 2025, 12:14 p.m.	1		CognitionPlatform		-			PASS	Ian	Things-Training

Figure 7.9 – Sessions

You will see the session and can confirm that datasets were added. Note the data also remains in the same local SavedData folder, so it is available even if there is a database crash or other problem.

Chapter 8

CageLab Tasks

Content

8.1. Task Architecture	39
8.2. Task Catalog	40
8.3. The theConductor Command API	43
8.4. cogmoteGO HTTP API	44

CageLab provides a library of behavioural tasks in the `+cltasks` package. Each task function is self-contained, following a standard pattern: initialise hardware, build stimuli, run a trial loop, log data, and clean up.

8.1. Task Architecture

All tasks share a common structure:

1. **Input parsing:** `clutil.checkInput(in)` validates and normalises parameters.
2. **Initialisation:** `clutil.initialise()` sets up `screenManager`, `audioManager`, `rewardManager`, `touchManager`, and logging.
3. **Stimulus construction:** Build opticka stimulus objects (`imageStimulus`, `discStimulus`, `metaStimulus`).
4. **Trial loop:** `while r.keepRunning ... end` — the core experiment logic.
5. **Data logging:** `clutil.updateTrialResult()` saves per-trial data.
6. **Cleanup:** `clutil.endTask()` closes hardware, saves final data, and optionally uploads to Alyx.

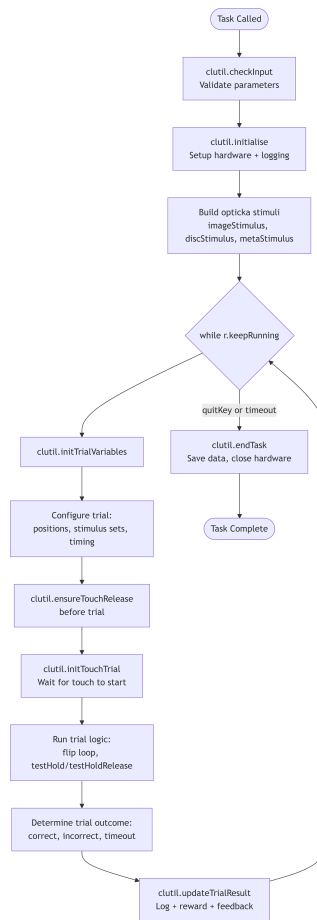


Figure 8.1 – General Trial Loop

8.2. Task Catalog

8.2.1. Touch Training (startTouchTraining)

The foundational task for teaching subjects to interact with the touchscreen. Uses an automated progressive shaping procedure through multiple phases:

Table 8.1 – Touch training

Phase Range	Parameter Shaped	Description
1–15	Size	Stimulus decreases from <code>maxSize</code> to <code>minSize</code>
16–20	Position	Stimulus appears at increasingly eccentric positions (3°, 5°, 7°, 11°)
21–32	Hold time	Required touch duration increases from 10ms to 400ms
33+	Release	Subject must touch, hold, then release; release window tightens from 3s to 1s

Parameters:

Table 8.2 – Touch parameters

Parameter	Default	Description
stimulus	'Picture'	'Picture' (flowers) or 'Disc'
fg	[0 0 0]	Stimulus foreground colour
minSize	2°	Minimum stimulus diameter
maxSize	10°	Maximum stimulus diameter
trialTime	5s	Maximum trial duration
easyMode	0	If 1, only first 20 phases (no hold/release)

8.2.2. Match-to-Sample (startMatchToSample)

A classic delayed match-to-sample (DMTS) or delayed non-match-to-sample (DNMTS) task. The subject views a sample stimulus, then after a delay must choose the matching (or non-matching) stimulus from an array of distractors.

Variants:

Table 8.3 – Match-to-sample variants

Variant	Description
MTS	Sample + distractors shown simultaneously; touch the matching one
DMTS	Sample shown, then delay, then distractors appear; touch the matching one
DNMTS	Sample shown, then delay, then distractors appear; touch the <i>non-matching</i> one

Parameters:

Table 8.4 – MTS parameters

Parameter	Default	Description
object	'fractals'	Stimulus category: 'fractals', 'quaddles', or 'flowers'
objectSize	10°	Stimulus diameter
objectSep	15°	Separation between stimuli
sampleY	0°	Vertical position of sample
distractorY	-10°	Vertical position of distractors
distractorN	2	Number of distractors (1–4)
sampleTime	1.0s	Sample viewing duration (scalar or [min max] range)
delayTime	1.0s	Delay between sample and choice (scalar or [min max] range)
delayDistractors	true	Show distractors during delay period
trialTime	5.0s	Maximum response window
targetHoldTime	0.2s	Required touch duration

Training mode: When `taskType` contains 'training', only one stimulus is shown (no distractors), hold time requirements are relaxed, and the delay is set to zero.

8.2.3. Intra/Extra-Dimensional Set Shifting (`startIED`)

Based on the CANTAB IED task, this measures cognitive flexibility and attentional set-shifting. Subjects learn to discriminate stimuli based on one dimension (e.g., shape) while ignoring another (e.g., colour), then must shift attention when the rule changes.

Stage Sequence (standard CANTAB order):

Table 8.5 – IED Types

Stage	Code	Description
Simple Discrimination	<code>sd</code>	Two shapes; learn which is rewarded
Compound Discrimination	<code>cd</code>	Shapes + colours added; shape remains relevant
Simple Reversal	<code>sr</code>	Same shapes; reward contingency reverses
Compound Reversal	<code>cr</code>	Same compound; reward contingency reverses
Intra-Dimensional Shift	<code>ids</code>	New exemplars; shape still relevant
Intra-Dimensional Reversal	<code>idr</code>	Same as IDS; contingency reverses
Extra-Dimensional Shift	<code>eds</code>	New exemplars; relevant dimension switches to <i>colour</i>
Extra-Dimensional Reversal	<code>edr</code>	Same as EDS; contingency reverses

Parameters:

Table 8.6 – IED parameters

Parameter	Default	Description
<code>taskType</code>	<code>'cd'</code>	Stage code (see table above)
<code>objectSize</code>	10°	Stimulus size
<code>objectSep</code>	15°	Separation between left/right targets
<code>sampleV</code>	0°	Vertical position
<code>trialTime</code>	5.0s	Response window
<code>targetHoldTime</code>	0.2s	Touch hold requirement

Stimuli: 6 unique shapes (circle, rect, triangle, heptagon, star, random) × 6 colours (red, green, blue, yellow, magenta, cyan), randomly paired per session.

8.2.4. Visual Oddball (`startVisualOddball`)

A simple go/no-go task: subjects respond to rare “deviant” stimuli and withhold response to frequent “standard” stimuli.

Parameters:

Table 8.7 – Visual Oddball parameters

Parameter	Default	Description
<code>deviantProbability</code>	0.2	Proportion of deviant trials
<code>objectSize</code>	8°	Standard stimulus size
<code>deviantSize</code>	8°	Deviant stimulus size
<code>stimulus</code>	'Disc'	'Disc' or 'Picture'
<code>standardColour</code>	[1 1 1]	RGB of standard (white)
<code>deviantColour</code>	[1 0 0]	RGB of deviant (red)
<code>standardShape</code>	'circle'	Shape name for picture mode
<code>deviantShape</code>	'triangle2'	Shape name for picture mode

8.2.5. Odd-One-Out / Things Task (`startThings`)

A visual oddity detection task. Three stimuli are shown; two are identical, one is different. The subject must touch the odd stimulus.

Parameters:

Table 8.8 – Things Task parameters

Parameter	Default	Description
<code>objectSize</code>	10°	Stimulus size
<code>objectSep</code>	15°	Separation between stimuli
<code>distractorSpreadAngle</code>	0°	Angular spread for polar layout (0 = horizontal line)
<code>distractorCenterAngle</code>	270°	Center angle for polar layout (270° = upward)
<code>pedestalOpacity</code>	varies	Highlight opacity for the odd target (training aid)
<code>trialTime</code>	5.0s	Response window

8.2.6. Drag Categorisation (`startDragCategorisation`)

A touch-and-drag task where subjects must move a stimulus to one of two target locations, testing categorisation or spatial matching.

8.2.7. IED Morphobes Variants

`startIEDmorphobes` and `startIEDmorphobes4D` extend the IED task with morphed stimulus continua, allowing parametric control of stimulus similarity. The 4D variant operates across four stimulus dimensions.

8.3. The theConductor Command API

`theConductor` exposes the following commands via ØMQ. These are used internally by the CageLab GUI but can also be sent programmatically:

Table 8.9 – Loop commands

Command	Data Required	Response	Description
<code>restruct('command','functionName', running_command {...})</code>			Execute a MATLAB function/command
<code>rundemo</code>	none	<code>demo_run</code>	Run PTB VBLSyncTest demo
<code>echo</code>	any	<code>echo_reply</code>	Echo back received data
<code>gettime</code>	<code>struct('GetSecs',value)</code>	<code>timesync_reply</code>	Round-trip timing test
<code>status</code>	none	<code>Processing</code>	Check if server is alive
<code>getlastrun</code>	none	<code>taskdata</code>	Retrieve <code>lastTaskRun.mat</code>
<code>commandlist</code>	none	List of commands	Get available commands
<code>syncbuffer</code>	<code>struct('frameSize',N)</code>	<code>syncbuffer_ack</code>	Set ØMQ frame size
<code>enablesleep</code>	none	<code>enable-sleep</code>	Enable display power saving
<code>disablesleep</code>	none	<code>disable-sleep</code>	Disable display power saving
<code>hidedesktop</code>	none	<code>hide-desktop</code>	Hide OS desktop
<code>showdesktop</code>	none	<code>show-desktop</code>	Show OS desktop
<code>exittask</code>	none	<code>invalid</code>	(only valid during task execution)
<code>exit / quit</code>	none	<code>bye</code>	Shut down command loop
<code>exitmatlab</code>	none	<code>bye</code>	Shut down loop and quit MATLAB

8.4. cogmoteGO HTTP API

`cogmoteGO` exposes a REST API on port `9012`:

Table 8.10 – REST API endpoints

Endpoint	Method	Purpose
<code>/api/health</code>	GET	Health check
<code>/api/status</code>	GET	Get running status + task info
<code>/api/cmds/proxies</code>	POST	Register a new command proxy
<code>/api/cmds/proxies</code>	GET	List all proxies
<code>/api/cmds/proxies/<name></code>	DELETE	Remove a proxy
<code>/api/broadcast/data/default/latest</code>	GET	Get latest broadcasted task data

Chapter 9

Troubleshooting

Content

9.1. Service Issues	45
9.2. Network Issues	46
9.3. Task Issues	47
9.4. Display Issues	48
9.5. Log Locations	48

This chapter covers common issues and their solutions.

9.1. Service Issues

9.1.1. cogmoteGO Won't Start

Symptom: `systemctl --user status cogmoteGO` shows `failed` or `inactive`.

Solutions:

```
# Check logs for error details
journalctl --user -u cogmoteGO.service -n 50 --no-pager

# Common fix: reset and reinstall
cagelab-stop
cogmoteGO service -u          # remove old service file
cogmoteGO service            # reinstall
systemctl --user daemon-reload
cagelab-start
```

9.1.2. theConductor Won't Start

Symptom: `systemctl --user status theConductor` shows `failed`.

Solutions:

```
# Check for MATLAB path issues
journalctl --user -u theConductor.service -n 100 --no-pager

# Verify MATLAB is installed and licensed
/usr/local/MATLAB/R2025b/bin/matlab -nodesktop -nosplash -r "exit"

# Verify ØMQ Java library
ls ~/Code/matlab-jzmq/

# Try manual start to see errors
cd ~/Code/CageLab-Code
matlab -nodesktop -nosplash -r "tc=theConductor('runNow',true);"
```

9.1.3. Service Dependencies Fail

The services start in a specific order. If one fails, subsequent services may also fail:

1. `toggleInput` should complete (oneshot)
2. `cogmoteGO` must be healthy (check: `curl http://localhost:9012/api/health`)
3. `theConductor` registers with `cogmoteGO` via HTTP API
4. `mediamtx` binds to media ports
5. `obs` connects to `mediamtx`

Use `cagelab-status` to verify the chain:

```
cagelab-status
# Should print: "CageLab service is running." plus broadcast data
```

9.2. Network Issues**9.2.1. Cannot SSH to Box****Solutions:**

1. Verify the box is powered on (try WoL: `ansible-playbook -e "wol_group=cagelab" playbooks/wol.yaml`)
2. Check NetBird connectivity: `netbird status`
3. Try both hostname and IP
4. Verify SSH key is correct: `ssh -vv cagelab@box-ip`

9.2.2. ØMQ Connection Fails

Symptom: CageLab GUI cannot connect to CageLab-Box.

Solutions:

1. Verify `cogmoteGO` is running on the box: `ssh box-host cagelab-status`
2. Check port accessibility: `nc -zv box-host 9012`
3. Check firewall: `sudo ufw status`

4. If using NetBird, ensure the VPN is up: `netbird status`

9.3. Task Issues

9.3.1. Task Crashes Mid-Session

Symptom: Task stops unexpectedly; data may be incomplete.

Data recovery: `theConductor` saves `~/lastTaskRun.mat` after each trial. The `getlastrun` command retrieves this:

```
% From MATLAB (via CageLab GUI's "Get Last Run" button)
```

Full session data is saved to `~/optickafiles/savedata/<date>/<subject>_<task>_<timestamp>/` and includes:

- `*.mat` — full MATLAB workspace
- `*.log` — text log of all events
- `info.json` — session metadata

9.3.2. Touchscreen Not Responding

Solutions:

```
# Check if touchscreen is disabled
xinput list | grep -i ilitek

# Enable it
toggleInput enable ILITEK-TP

# Verify it works
xinput test <device-id> # touch the screen; should see events
```

9.3.3. OBS Streaming Fails

```
# Check OBS status
systemctl --user status obs.service

# Restart OBS and MediaMTX
cagelab-reset-OBS

# Check OBS logs
journalctl --user -u obs.service -n 50

# Manually control recording
python3 ~/Code/CageLab-Code/scripts/control-OBS.py status --host 127.0.0.1
```

9.4. Display Issues

9.4.1. PTB Sync Failures

Symptom: PTB reports “Synchronization failure” or dropped frames.

Solutions:

1. **Ensure i3 is running** — GNOME/KDE compositors cause sync issues
2. **Disable display power saving:**

```
xset s off -dpms
```

3. **Set performance power profile:**

```
powerprofilesctl set performance
```

4. **Check display mode:** Use `xrandr` to verify refresh rate matches PTB expectations
5. **Run diagnostic:** Use the `rundemo` command from CageLab GUI to run PTB’s VBLSyncTest

9.5. Log Locations

Table 9.1 – Log locations

Log	Location
cogmoteGO	<code>journalctl --user -u cogmoteGO.service</code>
theConductor	<code>journalctl --user -u theConductor.service</code>
Task logs	<code>~/optickafiles/savedata/<date>/<session>/*.log</code>
OBS logs	<code>journalctl --user -u obs.service</code>
Task Starts	<code>~/cagelab-start.txt</code>
System logs	<code>journalctl</code> or <code>dmesg</code>
Task errors	Messages appended to <code>~/cagelab-start.txt</code>

Appendix A

Appendix

Content

A.1. File and Directory Layout	49
A.2. systemd Service Dependency Graph	50
A.3. Network Port Reference	51
A.4. Key Environment Variables	51
A.5. CageLab Version History	51
A.6. Further Reading	51

A.1. File and Directory Layout

```
~/Code/
├─ CageLab-Code/           # Main CageLab repository
│   ├── +cltasks/         # Behavioural task functions
│   ├── +clutil/          # Shared utility functions
│   ├── ansible/          # Ansible playbooks and inventory
│   ├── docs/             # Documentation and images
│   ├── scripts/          # ZSH convenience scripts
│   ├── services/         # systemd unit files
│   ├── setup/            # Bootstrap and configuration
│   │   ├── bootstrap.sh  # Initial machine setup
│   │   ├── makelinks.sh  # Symlink management
│   │   └─ config/        # Config files (zsh, i3, tmux, ssh, etc.)
│   └─ theConductor.m     # ØMQ REP server class
├─ Psychtoolbox/          # Psychtoolbox-3
├─ opticka/               # Core stimulus/task framework
├─ matmoteGO/             # MATLAB ØMQ client
├─ PTBSimia/              # Hardware interfaces (reward pumps)
├─ matlab-jzmq/           # Java ØMQ bindings
└─ PacmanTask/            # Task-specific code

~/bin/                    # Symlinks to scripts/*.sh
```

```
~/.config/systemd/user/    # Symlinks to services/*.service
~/.config/tmuxp/           # tmuxp configuration
~/.config/rsync-excludes   # rsync exclusion patterns
~/.ssh/config              # SSH configuration (symlinked)
```

A.2. systemd Service Dependency Graph

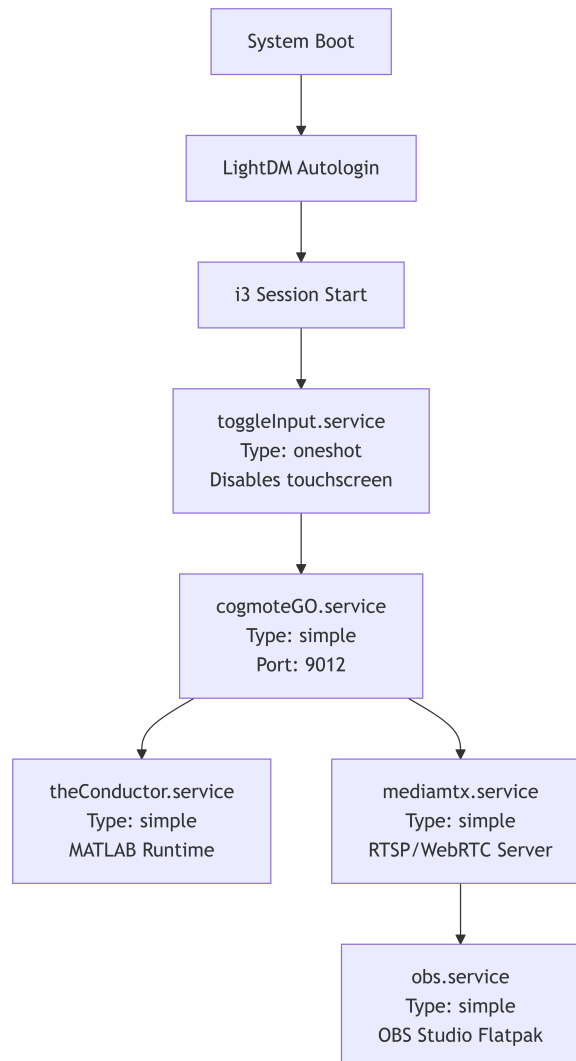


Figure A.1 – systemd boot dependencies

A.3. Network Port Reference

Table A.1 – Ports used

Port	Service	Protocol	Purpose
22	SSH	TCP	Secure shell access
4000	NoMachine NX	TCP	Remote desktop
6666	theConductor	ØMQ (internal)	REP server bound port
9012	cogmoteGO	HTTP	REST API + command bridge
8554	MediaMTX	TCP	RTSP streaming
8889	MediaMTX	TCP	WebRTC signalling
8189	MediaMTX	UDP	WebRTC ICE
4455	OBS Studio	TCP	WebSocket control API
51820	NetBird	UDP	WireGuard tunnel

A.4. Key Environment Variables

Table A.2 – ENV vars

Variable	Purpose
NETBIRD_SETUP_KEY	Provision new NetBird peers
OBS_WS_PASSWORD	OBS WebSocket authentication
chrony_master_ip	NTP master for chrony_slave playbook
DISPLAY	X11 display target (usually :0)
XDG_RUNTIME_DIR	systemd user session runtime

A.5. CageLab Version History

Table A.3 – Versions

Version	Date	Key Changes
1.0.70	2025-07	Test fixes, SSH config updates
1.0.0	2025-Q1	Initial stable release

A.6. Further Reading

- **opticka Framework:** github.com/iandol/opticka
- **Psychtoolbox-3:** psychtoolbox.org/ 
- **ZeroMQ Guide:** zguide.zeromq.org/
- **Ansible Documentation:** docs.ansible.com/
- **NetBird Docs:** docs.netbird.io/
- **NoMachine:** www.nomachine.com/
- **MediaMTX:** github.com/bluenviro/mediamtx
- **OBS Studio:** obsproject.com/

Appendix B

Index

A

Alyx	6, 36, 39
Ansible	14
Group Maintenance	5, 25, 26
Inventory	5, 25
Playbooks	22

Audio

AudioManager	16
--------------	----

C

CageLab	13
CageLab GUI	15
Usage	6, 35, 36, 38
Cagepull	6, 33
CANTAB	
IED Task	6, 46
CogmoteGO	
HTTP API	6, 43, 48
Control-PC	
Installation	19

G

Gitee	20
-------	----

I

I3 Window Manager	17
ILITEK-TP	16
Installation	5, 17, 18, 20

L

LightDM	24
Logging	
Locations	7, 49, 52

M

Makelinks.Sh	18
MATLAB	18
Toolboxes	19
Matlab-Jzmq	20
MatmoteGO	20
MediaMTX	15

N

NetBird	5, 15, 21, 22, 23
Netbird	18
Network Setup	5, 21, 22, 24
NoMachine	5, 21, 23

O

OBS Studio	15
Control-OBS.Py	32
Opticka	20

P

PacmanTask	20
Pixi	18

Ports		Troubleshooting	7, 49, 50, 52
Network Reference	7, 53, 55	PTB Sync	7, 52
Power Management	52	V	
Psychtoolbox	14		
Psychtoolbox-3	20		
PTB, see—Psychtoolbox	14	Video Recording, see—OBS Studio	16
PTBSimia	16, 20	VPN, see—NetBird	5, 21, 22, 23
R		W	
Reward		Wake-On-LAN	26
RewardManager	43	Configuration	26
S		Z	
Screen		ZeroMQ(O MQ)	14
ScreenManager	43		
VSync	7, 52		
Services			
Managing	5, 29, 30, 32		
SSH	5, 21		
Key Generation	5, 22		
Stimuli			
DiscStimulus	43		
ImageStimulus	43		
MetaStimulus	43		
Systemctl	6, 29, 31		
Systemd	14		
Dependency Graph	7, 53, 54		
User Services	29		
T			
Tasks	6, 43, 44, 46, 48		
Drag Categorisation	6, 47		
IED	6, 46		
IED Morphobes	6, 47		
Match-To-Sample	6, 45		
Odd-One-Out	6, 47		
Touch Training	6, 44		
Visual Oddball	6, 46		
TheConductor			
Command API	6, 43, 47		
Tmux	31		
Tmuxp	31		
Touchscreen	16		
Management	6, 29, 32		
TouchManager	16		